INTERVIEWER GUIDE

AI Session: Tally

What this session measures

A two-hour session on a small codebase the candidate has never seen, with an AI assistant of their choice. They first read the code with the assistant and answer our questions about its modules, its functions and its flow. Then they fix one bug we tell them about, build one or two features you pick from a menu, and chase a bug report whose causes we do not tell them. Whether they finish does not matter much. What matters is how they work with the assistant, and whether they notice when the assistant is wrong.

Why this codebase

Tally keeps a shared expense ledger for a group of people: who paid for what, who owes whom, and which payments would settle everything. Everybody has split a restaurant bill, so nobody needs the domain explained, and anybody can check an answer with a pen. There is no framework, no database and no network: a command-line program that reads one command per line, about 300 to 450 lines of code plus tests, using only the standard library (and JUnit through Maven for the Java tests).

The same program exists in **Python**, **Go** and **Java**, with the same file layout, the same output byte for byte, and the same bugs in the same places. The candidate picks the language. Their evidence is comparable across languages because the tasks are.

Small and familiar does not mean easy. Money is kept as integer cents, so every split has to decide where the leftover cent goes. Balances are kept up to date as entries come in, and the settle plan works on them directly. An AI assistant reads this code in seconds and explains it confidently; it is also confidently wrong about some of it. The candidate can check any claim by running the program on a three-line input, and whether they do is the main thing we watch.

The code carries six bugs that the tests do not catch. All six were reproduced in all three languages before this guide was written; the outputs below are copied from real runs.

	Bug	Where	Effect	Surfaces in
B1	Equal split drops the leftover cents	split · `equal`	100.00 among three gives 33.33 each; balances add up to +0.01 and `settle` never clears it	Task 1 (reported)
B2	Shares split drops them too	split · `by_shares`	The same leak, for weighted splits	Task 1, if they look
B3	One-digit cents read as cents	money · `parse_amount`	`12.5` is recorded as 12.05; `1.234` is accepted as 3.34	Task 3
B4	Expense ids reused after a delete	ledger · `add_expense`	The new id is `len + 1`, so two expenses share an id and `delete` removes the older one	Task 3, F4
B5	Exact parts may add up to less than the amount	split · `exact`	The check is > instead of ≠; the missing money is silently credited to the payer	Invariant test, F1
B6	`settle` changes the live balances	ledger · `balances`, settle · `plan`	`balances` returns the ledger's own map and the plan drains it: after `settle`, everyone shows 0.00	F3, or `settle` then `balances`

Names follow Python; Go and Java use their own casing (`ByShares`, `byShares`), in the matching file. The README promises that "the balances of a group always add up to zero". B1, B2 and B5 break that promise; a candidate who turns it into a test finds all three at once.

Format

Individual, about 120 minutes a reading phase, three tasks, then a 15 minute debrief. Book a two-hour slot.

Read first, then questions the session opens with 10 minutes of reading the code with the AI, then 10 minutes of questions about its modules, its functions and the flow of a command through them, with the AI closed. Nobody changes code before the questions are done.

Talk while the AI works when the candidate is waiting for the AI to generate code, use the time to talk about their history, their projects and their way of working. Stop as soon as the output arrives. See "While the AI works" below.

Their own AI any tool is allowed; Claude Code, Cursor, Copilot, or a plain chat window. Note which one it was and whether it could run commands by itself. Note before the session which skills, MCP servers and rules files (`CLAUDE.md`, `AGENTS.md`, Cursor rules) are configured. A web search or GitHub MCP server can find public code, and this repository is public and contains this guide: ask the candidate to turn web and GitHub access off for the session, or note that it stayed on.

Their own language Python, Go or Java. Ask in advance, so the environment is ready.

Observed one interviewer with a notes sheet, plus screen recording. Without an observer, the AI transcript and the git history keep most of what you need.

Commit after every task the candidate tags the end of each task (`task-1`, `task-2`, …). Unfinished work is fine and is evidence in itself.

Tasks are handed out one at a time Task 3 is easier for someone who already saw B1 in Task 1, and the feature you choose may surface B4 or B6. Handing tasks out early spoils that.

Preparation

- ☐ Copy `ai-session/tally/<language>/` from this repository into a **new, private** repository with a neutral name (for example `tally`) and a single initial commit. Do not link to this repository: it is public and contains this guide.
- ☐ Do **not** fix anything. Every task depends on the bugs being there.
- ☐ Python: 3.12 or later, nothing to install. Go: 1.22 or later. Java: JDK 21 and Maven; run `mvn -q package` once on the machine so JUnit is already in `~/.m2`.
- ☐ Confirm the setup: the tests pass, the example in the README prints exactly what the README shows, and the TAL-12 input below shows `ali +66.67`.
- ☐ Choose the feature(s) for Task 2 from the menu, according to the level you are hiring for and the time you expect to have.
- ☐ Print this guide and write each brief on a separate card. Task 3 needs Reza's transcript on its card.

What you hand to the candidate

> **Read verbatim:** You are joining the team that maintains Tally, a small program that splits shared expenses. You have never seen this code. You have about two hours and an AI assistant of your choice.
>
> 1. Use the AI as much or as little as you like. We care about how you work, not about whether you use it.
> 2. Think aloud. Tell us what you ask the AI and why, and what you conclude after it answers.
> 3. After each task, commit and tag (`git tag task-N`). Unfinished is fine.
> 4. If something in a task is unclear, you may ask us. We may or may not answer.
> 5. Keep your AI transcript. We collect it at the end.
> 6. While your AI is generating code, we may talk about your background and how you work. Go back to the output whenever you want; you do not need to finish the sentence.

While the AI works

An agentic tool often works for a minute or more before it shows anything. Do not spend that time in silence: talk with the candidate about their history, their projects and their way of working. It fills dead time, and it gives us a second view of the person next to the one the tasks give.

Rules:

- Talk only while the candidate is **waiting**. Never interrupt them while they are reading output, writing a prompt or thinking aloud about the code.
- Stop as soon as the output arrives, even mid-answer. What the candidate does next is evidence: do they turn to the output and read it, or keep talking and accept it unread? Note which.
- Do not talk about the task, the bugs or the code. That is what the debrief is for, and a hint here spoils the tasks.
- The time boxes do not stop for the conversation. Keep the questions short, and pick up a topic again at the next wait instead of finishing it.
- Ask only about work. No questions about age, family, health, religion or anything else that has no place in an interview.

Pick from these as the waits come; you will not get through all of them.

History How did you get into software? Which job or project taught you the most, and what did it teach you? What is a bug you chased for a long time, and how did you find it in the end?

Projects Tell us about a system you work on now. What is your part of it? Which decision in it would you make differently today? What did you build that you are proud of, and what did it take?

Way of working How do you use an AI assistant in your daily work, and for what do you not use it? When did an AI last give you something wrong, and how did you notice? How do you review a teammate's code, and what do you look at first? When do you write tests: before, during or after? How do you pick up a codebase you have never seen?

Setup Which skills and MCP servers have you configured, and why those? Is there one you wrote yourself? Show it. Which did you try and then remove, and why? What is in your rules file (`CLAUDE.md`, `AGENTS.md`, Cursor rules)? Which of them did your AI use today, and did it help?

A setup is not scored by its size. A candidate with a plain tool and no configuration loses nothing. What counts is whether they know what their setup does and why it is there: a skill they cannot explain, or an MCP server they did not know was on, is worth a note.

Score this conversation in the same four competencies as everything else; do not give it a score of its own. Most of it is evidence for **effective communication**: can they explain a system they know to someone who does not? An answer about their way of working that the session itself contradicts (says they always read the diff, then accepts output unread) is worth a note too.

READING Read the code

20 minutes

BRIEF FOR THE CANDIDATE

Read the code with your AI assistant, in any way you like: ask it for an overview, go through the files together, run the tests and the example from the README. Do not change any code. You have 10 minutes.

Then we close the AI and ask you about the modules, the functions, and how a command flows through them. The code stays open, and you may look at it while you answer.

No trap. This is the baseline for how the candidate reads with the AI, and the questions show what they took away from it: their own picture of the code, or the assistant's summary. For the questions, the AI window is closed or minimised; the editor stays open. Finding the answer in the code is a good answer. Guessing from what the AI said, without looking, is the thing we want to see.

Ask six or seven of the questions below: always 1 and 4, and at least one flow question (11 or 12). After each answer ask "where is that?" and let them point at the code. Do **not** say whether an answer is right, and do not correct it: Tasks 1 to 3 depend on the candidate finding the bugs themselves. If they name a bug now, write it down and move on.

	Question	What a good answer says
1	What are the modules, and what does each one do?	`money` parses and formats amounts as integer cents; `split` turns a split specification into the cents each member owes; `ledger` keeps members, expenses, payments and balances; `settle` works out the transfers; `cli` reads one command per line, calls the others and prints the results.
2	Which module uses which?	`cli` uses all four; `split` uses `money`; `ledger` and `settle` use nothing from the project. The ledger never sees a split specification, only the computed shares (this matters for F4).
3	How is an amount kept, and why?	As integer cents, so every sum is exact. Decimals exist only at the edges, in `parse_amount` and `format_amount`.
4	Where is a member's balance stored, and which functions change it?	One map in the ledger, from name to cents. `add_member` sets it to zero; `add_expense` and `delete_expense` change it through `_apply` with a sign of $+1$ or -1; `record_payment` changes it; and so does `settle.plan`, through the map that `balances()` returns. Only a candidate who read `plan` says the last part. **Touches B6.**
5	What does `parse_split("shares:ali=1,sara=3",`	`{ali: 250, sara: 750}`. No, the payer does not have to be in the split.

	Question	What a good answer says
	`1000)` return? Must the payer be in the split?	
6	What does `delete_expense` do?	It finds the first expense with that id, removes it, and reverses its effect with `_apply(expense, -1)`; with no match it raises an error. **Touches B4.**
7	How does `plan` choose each transfer?	The member who owes the most pays the member who is owed the most, the smaller of the two amounts; ties by name; repeat until nobody is owed or nobody owes. It is greedy and does not promise the fewest transfers (F5).
8	What happens when one command in a file fails?	`run` prints `error: line N: ...`, counts the failure and goes on with the next line; the program exits with 1 if anything failed. Blank lines and lines starting with # are skipped.
9	What does `parse_amount` return for `7` and for `12.50`?	700 and 1250. Do not ask about `12.5`: that is B3, and Task 3 needs it.
10	How does an expense get its id?	`len(expenses) + 1`. **Touches B4.**
11	Flow: follow the line `expense ali 30.00 equal:ali,sara cab` from the input to the printed `#1`.	`run` strips the line and skips blanks and comments; `execute` splits it into words and picks the `expense` branch; `parse_amount` gives 3000; `parse_split` calls `equal`, which gives 1500 each; `add_expense` checks the members with `_require`, builds the `Expense`, appends it and calls `_apply`, which adds 3000 to Ali and takes 1500 from each member of the split; `execute` prints the line with `format_amount`.
12	Flow: what happens when the user types `settle`?	`execute` checks the arity, calls `ledger.balances()`, hands the map to `settle.plan`, and prints each transfer or "nothing to settle". The ledger records nothing. A candidate who followed the map into `plan` adds that the plan empties the ledger's own balances. **Touches B6.**

Names follow Python; ask with the names of the language the candidate chose.

What to watch for:

- While reading: do they only ask the AI for a summary, or do they open files, ask for line references, and run the tests and the example?
- Are the answers specific (a file, a function, a field) or general ("it handles the money")? Does the candidate look at the code before answering, or answer from memory of the AI's summary?
- Wrong answers that repeat a claim the AI made are the main evidence here. Note each one; they count as AI claims accepted without checking.
- "I don't know, let me look" is a good answer. A confident guess is not.

TASK 1 The missing cent (known bug)

25 minutes

BRIEF FOR THE CANDIDATE

Ticket **TAL-12**, from a user:

"Ali paid 100.00 for a dinner with Sara and Reza and we split it equally. `balances` shows ali +66.67, reza −33.33, sara −33.33. That adds up to +0.01. Sara and Reza paid what `settle` told them to, and Ali is still owed 0.01 by nobody."

Fix it, with a regression test that fails before the fix and passes after it.

Groups disagree about who should receive the leftover cents, so make it configurable. Two rules are asked for today: **the payer first**, and **in the order the split lists the members**. Adding a third rule later must be easy. Choose a default, tell us which one, and test each rule.

```
member ali
member sara
```

```
member reza
expense ali 100.00 equal:ali,sara,reza dinner
balances
```

Trap · The bug the ticket shows is not the only copy of it. `equal` gives everyone `amount // n` and drops the rest. Fixing that line closes the ticket. But `by_shares` does the same with `amount * weight // total` (B2): `10.00` with `shares:ali=1,sara=1,reza=1` leaks a cent in exactly the same way. The root-cause fix is a single allocation function that both splits use, with `equal` as shares of 1.

The configurable rule makes the same point harder to miss: if only `equal` takes the rule, a `shares` split still ignores it. With one allocation function, the rule is applied in one place.

Trap · the split does not know the payer. `parse_split(spec, amount)` gets the specification and the amount, nothing else. The rule "payer first" needs the payer, and the chosen rule has to come from somewhere. So the candidate has to decide where the setting lives (the ledger, a `leftover <rule>` command, an option on each expense, a command-line flag) and how it reaches the split. There is no single right answer. We want to see the decision made on purpose, stated, and ideally asked about. Two smaller decisions follow from it: what "payer first" means when the payer is not in the split, and what a rule change does to expenses already recorded. The ledger stores the computed shares, so old expenses keep their cents; a candidate who says so has read `add_expense`.

Go only · the cent moves between runs. Shares are a `map`. The obvious fix, "loop over the shares and add one cent until nothing is left", walks the map in random order, so a different person gets the cent on each run. We reproduced it: eight runs of the TAL-12 input printed `ali +66.66` three times and `ali +66.67` five times. A test that asserts who got the cent then fails some of the time. A strong Go candidate sorts the names before handing out cents, or notices the flaky test and asks why.

The rule "in the order the split lists the members" makes this worse: `parsePairs` returns a map, so for a `shares` split the listed order is lost before `ByShares` is called. The fix has to keep the order, for example as a slice of names next to the map. Python dicts and Java's `LinkedHashMap` keep it already.

What to watch for:

- Does the regression test fail before the fix? The AI often asserts that the three shares are "33.33, 33.33, 33.34" without caring who gets the 33.34; it also often writes a test that only checks `33.33` and passes on the buggy code.
- Floating point: the AI sometimes proposes `round(amount / 3, 2)` or a decimal type. That does not fix anything; the cents are already integers. Does the candidate see that?
- Do they look at `by_shares` (and `exact`) without being asked? Do they turn the README's "always adds up to zero" into a test? A test like that, run against a few splits, also finds B5.
- Is the default rule stated, or chosen silently by the AI?
- How big is the change? The AI likes a strategy interface, a registry, a configuration file or an environment variable. A rule name and one function that orders the names is enough; adding a rule is then one more branch. Does the candidate keep the diff in proportion?
- Is the rule checked before it is stored? `leftover banana` should be an error, not a silent fallback to the default.
- Does each rule have a test that tells it apart from the others? On the TAL-12 input all three reference rules give the cent to Ali, so a test on that input alone proves nothing about the rule. `expense sara 100.00 equal:reza,ali,sara` separates them: the extra cent goes to Ali by largest remainder (ties by name), to Sara with "payer first", and to Reza with "listed".

TASK 2 Features (interviewer's choice)

35 minutes

Pick one feature, or two short ones, from the menu. The level describes the nature of the work, not the job title. Each feature is designed to bring a different part of the code into play; the last column says which bug it is likely to surface. Tell the candidate only the brief.

	Feature	Level	Time	Surfaces
F1	`percent` split	1	15 min	B5, B1 again
F2	`history <member>`	1	15 min	B1, B2
F3	`settle --record`	2	20 min	B6
F4	`edit <id> <amount>`	2	20 min	B4
F5	Fewest transfers	3	35 min	B6

F1 Percent split

Level 1 · 15 minutes

BRIEF FOR THE CANDIDATE

Add a split kind `percent:ali=50,sara=30,reza=20`. The percentages must add up to exactly 100; anything else is an error. Tests included.

What to watch for:

- Percent is shares with weights out of 100. Does the candidate reuse the allocation from Task 1, or does the AI write a fourth loop with its own rounding?
- The AI usually copies the validation from `exact`, including its > check. A candidate who reads the copy, or who wrote the zero-sum test, finds B5 here.

F2 History of a member

Level 1 · 15 minutes

BRIEF FOR THE CANDIDATE

Add `history <member>`: every expense and payment that involves the member, in the order they were recorded, with the amount it changed their balance by, and the balance at the end. The lines must add up to what `balances` shows.

What to watch for:

- The ledger keeps each expense's shares but no per-member log, so the history has to be rebuilt from the expenses and payments. Does the candidate know where those live without the AI saying so?
- "The lines must add up" is a check the candidate can run. If B1 or B2 is still there, the history of the payer is off by the leftover cent. Who notices, the candidate or nobody?
- Deleted expenses must not appear. Payments are stored but never shown anywhere today; does the candidate find them?

F3 Record the settlement

Level 2 · 20 minutes

BRIEF FOR THE CANDIDATE

Add `settle --record`. It prints the same plan as `settle`, and also records each transfer as a payment, so that `balances` afterwards shows everyone at zero.

Trap · zero, then minus. `plan` drains the map that `balances()` returns, and that map is the ledger's own (B6). After the plan is computed, every balance is already zero; recording the payments then moves each one **past** zero. On a single hotel expense the result is exactly the original balances with the sign flipped:

```
before   {ali: +60.00, sara: -30.00, reza: -30.00}
after    {ali: -60.00, sara: +30.00, reza: +30.00}
```

The AI often "fixes" this by skipping the payments, since the balances were already zero. That makes the output look right and leaves the bug in place: no payments are recorded, and plain `settle` still wipes the balances. The real fix is that `balances()` returns a copy, or that `plan` copies its input. Both are fine; which one, and why, is a good debrief question. Note that `expenses()` and `payments()` already return copies; the inconsistency is visible in the file.

F4 Edit an expense

Level 2 · 20 minutes

BRIEF FOR THE CANDIDATE

Add `edit <id> <amount>`: change the amount of an expense and keep its split. Balances must be as if the expense had been recorded with the new amount in the first place.

What to watch for:

- "Keep its split" is ambiguous: the ledger stores the computed shares, not the split specification. Rescaling the shares is wrong for `exact` and rounds differently from a fresh split. Does the candidate notice and ask, or store the specification, or let the AI choose?
- B4: `edit` finds an expense by id. After any delete and a new expense, two expenses share an id. A candidate who tests `delete` then `expense` then `edit` sees the wrong one change.

F5 Fewest transfers

Level 3 · 35 minutes

BRIEF FOR THE CANDIDATE

`settle` sometimes suggests more payments than needed. For balances `ali +60.00, sara +50.00, reza -20.00, mina -40.00, omid -50.00` it suggests four; three are enough. Make `settle` suggest the smallest possible number of payments, for groups of up to 12 members.

```
omid → ali 50.00      the greedy plan     omid → sara 50.00     a shortest plan
mina → sara 40.00                         mina → ali 40.00
reza → ali 10.00                          reza → ali 20.00
reza → sara 10.00
```

What to watch for:

- The minimum is $n - k$, where k is the largest number of disjoint groups whose balances add up to zero. That is a subset problem; a bitmask DP over the 2^n subsets is fast enough for 12 members. A candidate who sees "partition into zero-sum groups" without the AI is showing Level 3 problem solving; one who gets the idea from the AI should still explain why it is correct.
- The AI often answers with another greedy heuristic ("match equal amounts first") and a test on the example only, which passes. Does the candidate ask for a counterexample, or write a brute-force checker to compare against?
- The candidate needs a ledger that produces these balances; `exact` splits do it in two lines. And if B6 is still there, every experiment that runs `settle` twice gets nothing the second time.

TASK 3 Reza's weekend (unknown causes)

25 minutes

BRIEF FOR THE CANDIDATE

Ticket **TAL-31**. Reza used Tally interactively and sent us what they typed and what they saw:

```
member ali
member sara
member reza
expense ali 60.00 equal:ali,sara,reza groceries
#1 groceries: ali paid 60.00
expense sara 12.5 equal:sara,reza taxi from the station
#2 taxi from the station: sara paid 12.05
expense reza 45.00 equal:ali,sara,reza museum tickets
#3 museum tickets: reza paid 45.00
delete 1
deleted #1 groceries
expense ali 30.00 equal:ali,sara,reza fuel
#3 fuel: ali paid 30.00
delete 3
deleted #3 museum tickets
balances
ali   +20.00
reza  -16.02
sara  -3.97
```

“The hotel paid for the groceries and the fuel was a duplicate, so I deleted both. I paid 45.00 for the museum, so the group owes **me**. By my count it is ali −15.00, sara −8.75, reza +23.75. The tool says I owe 16.02.”

Find out what went wrong, fix what is broken in Tally with regression tests, and write the reply you would send to Reza in two or three sentences.

Trap · two bugs, and a replay that still disagrees.

- **B3**: `12.5` is parsed as 12 whole and 5 cents. Reza typed a valid amount; the tool printed `12.05` right back and nobody looked.
- **B4**: after `delete 1`, the next id is `len + 1 = 3`, which the museum already has. `delete 3` removes the first match, the museum. The output shows `#3` twice.
- If B1 is still unfixed, the taxi (now 12.05 between two people) also leaks a cent; that is why the three balances add up to +0.01.
- Reza’s own numbers are right. With both bugs fixed and the fuel deleted by its new id, the tool prints exactly ali −15.00, reza +23.75, sara −8.75.
- The final trap: replaying the transcript **literally** after the fix still deletes the museum, because the fuel is now `#4` and the transcript says `delete 3`. The numbers are still wrong (ali +20.00, reza −16.25, sara −3.75). A strong candidate explains that Reza typed 3 because the tool printed 3, and replays with 4.

What to watch for:

- Do they replay the transcript as a file and compare it line by line with what Reza saw, or ask the AI to guess? The `12.05` and the second `#3` are both visible in the output.
- Expect the AI to say that `12.5` is user error (“amounts need two decimals”), and that the missing 0.01 is rounding. Both claims are wrong. Does the candidate push back with evidence?
- For B4, does the fix give every expense an id that is never reused, or does the AI change `delete` to remove the last match or all matches? Those only move the problem.
- B3: is `1.234` now rejected, or silently read as 1.23? Either is defensible if it is stated; `3.34` is not.
- Is the reply to Reza honest and specific, or does it blame Reza?

Debrief (15 minutes)

1. Name every place in the code that changes a balance. Which of them did you not expect?
2. Which of the AI’s claims turned out to be wrong today? How did you find out?

3. Which leftover rule is the default, and who decided that? The group switches rules after a week of expenses: what happens to the expenses already recorded?
4. Reza has saved command files from last year and replays them. Does your id fix change what any of them mean? (Yes: a file that deletes, adds, and then deletes by the id the old tool printed now targets a different expense. A candidate who sees this understands what they changed.)
5. Which change would you merge with the least confidence, and why?
6. What single check would you add to the tests so that this family of bugs cannot come back? (The balances add up to zero after every command.)

Scoring

Score the session with the **same four competencies** and the **same scale** as the evaluation document: **Weak · Borderline · Strong · Not assessed**. Do not invent a per-task score. Each competency gets one rating for the whole session, backed by the evidence you wrote down; a competency you saw no evidence for is **Not assessed**, not a guess. Write next to the ratings which features you chose and their level, because Strong on a Level 1 feature and Strong on F5 are different evidence.

Competency	Weak	Borderline	Strong
Data structures, algorithms and problem solving	Patches each symptom where it appears; cannot say at the debrief where balances live or why the cent went missing	Finds the root cause of the reported bug but not its copies; needs a nudge to look at `by_shares` or the second `#3`	Treats the zero-sum promise as an invariant; finds sibling bugs unprompted; explains B4 and B6 from the data structures, not from the symptom
Coding	Lets the AI rewrite modules or add libraries; tests pass before the fix too; leaves a flaky test in place	Working fixes, but duplicated logic (a fourth rounding loop) or tests that do not pin the behaviour	Small diffs in the style of the repository; one allocation function; each regression test fails first; deterministic in every language
Effective communication	Prompts like “the cents are wrong, fix it”; accepts “user error” and “rounding” as explanations; decisions made silently by the AI	Explains what they are doing but leaves the extra-cent rule or the edit semantics implicit	Prompts carry the input, the output and the expectation; challenges a wrong claim with a run; names each decision and asks or states the assumption; the reply to Reza is clear and honest
Speed and getting to a result	Applies AI suggestions one after another until the output looks right; never replays Reza’s transcript	Gets there, but slower than the time boxes because checks come late	Reproduces before fixing, verifies after, and still finishes Tasks 1 and 3 and the chosen feature in time

Fill in the bug sheet as you go. It is not a score; it is the evidence the ratings point back to.

	Found by	Root cause fixed?	Test failed before fix?	Notes
B1	candidate / AI / no			
B2	candidate / AI / no			
B3	candidate / AI / no			
B4	candidate / AI / no			
B5	candidate / AI / no			
B6	candidate / AI / no			

Also record three plain counts per task: **files the candidate opened**, **files the AI opened**, and **AI claims accepted without checking**. And note when the candidate asked the AI to “find all the bugs”: that is a legitimate move. What counts is what they did with the list, which on this codebase usually mixes real bugs with confident false ones.

Collecting data

Collect the git history with the `task-N` tags and the final working tree; the AI transcript (a Claude Code session export, the Cursor chat, or the text of the chat window); the screen recording with the candidate's audio; and the interviewer's sheet with timestamped notes and the bug sheet.

Reference fixes

A short version for the interviewer, in Python. The same fixes were applied to all three languages: all existing tests still pass, the three versions agree on every input in this guide, and TAL-31 replayed with `delete 4` prints Reza's numbers.

B1 and B2 · one allocation, a configurable rule

```python
# split.py
RULES = ("remainder", "payer", "listed")

def equal(amount, names, payer, rule):
    return by_shares(amount, {name: 1 for name in names}, payer, rule)

def by_shares(amount, weights, payer, rule):
    total = sum(weights.values())
    shares = {name: amount * w // total for name, w in weights.items()}
    left = amount - sum(shares.values())
    for name in _leftover_order(amount, weights, payer, rule)[:left]:
        shares[name] += 1
    return shares

def _leftover_order(amount, weights, payer, rule):
    names = list(weights)  # in the order the split lists them
    if rule == "listed":
        return names
    total = sum(weights.values())
    # largest remainder first, ties by name
    by_remainder = sorted(names, key=lambda n: (-(amount * weights[n] % total), n))
    if rule == "payer":
        # stable sort: the payer first if in the split, then the rest as before
        return sorted(by_remainder, key=lambda n: n != payer)
    return by_remainder
```

```python
# ledger.py, in __init__
self.leftover = "remainder"

# cli.py, in execute()
elif command == "leftover":
    _arity(args, 1, "leftover remainder|payer|listed")
    if args[0] not in RULES:
        raise ValueError(f"unknown leftover rule: {args[0]}")
    ledger.leftover = args[0]
...
shares = parse_split(spec, amount, payer, ledger.leftover)
```

`parse_split` passes `payer` and `rule` on to `equal` and `by_shares`. The leftover is always less than the number of members, so every member gets at most one extra cent. With the default rule TAL-12 gives ali +66.66 (Ali, first by name, carries the extra cent of the dinner Ali paid for), reza −33.33, sara −33.33; the other two rules give the same on this input. In Go, keep the listed order in a slice next to the map and sort with `sort.SliceStable`; in Java, a stable `Comparator` on the remainder, then the name.

B3 · at most two decimals

```python
# money.py
_AMOUNT = re.compile(r"[0-9]+(\.[0-9]{1,2})?")
...
cents = int(whole) * 100 + int(fraction.ljust(2, "0"))
```

B4 · ids that are never reused

```
# ledger.py
self._next_id = 1                       # in __init__
...
expense = Expense(self._next_id, payer, amount, shares, description)
self._next_id += 1
```

B5 · exact parts must add up

```
# split.py, in exact()
if total ≠ amount:
    raise ValueError(f"exact parts add up to {format_amount(total)}, not
{format_amount(amount)}")
```

B6 · do not hand out the ledger's own map

```
# ledger.py
def balances(self) → dict[str, int]:
    return dict(self._balances)
```

In Go, `maps.Clone(l.balances)`; in Java, `new LinkedHashMap<>(balances)`. Copying inside `plan` instead is equally correct.

Caveats

- Verified with Python 3.15, Go 1.27 and Temurin JDK 21 with Maven 3.9.9 and JUnit 5.11. The code uses nothing newer than Python 3.12, Go 1.22 and Java 21.
- The code is small enough that an agentic AI can read all of it in one pass. That is intended: the session is about verification, not search. Expect "find the bugs" prompts and judge what follows.
- The code lives in a public repository. If a candidate could plausibly have seen it, prefer another session.
- The time boxes assume a prepared environment. For Java, a first Maven download can take minutes; do it before the session.